CONSTRUCTOR UNIVERSITY

Object specific distance estimation techniques for autonomous driving

by

Timofei Podkorytov

Bachelor Thesis in Computer Science

Submission: May 19, 2026 Supervisor: Prof. J. Suchan

Constructor University | School of Computer Science and Engineering

Statutory Declaration

Family Name, Given/First Name	Podkorytov, Timofei
Matriculation number	30007142
Kind of thesis submitted	Bachelor Thesis

English: Declaration of Authorship

I hereby declare that the thesis submitted was created and written solely by myself without any external support. Any sources, direct or indirect, are marked as such. I am aware of the fact that the contents of the thesis in digital form may be revised with regard to usage of unauthorized aid as well as whether the whole or parts of it may be identified as plagiarism. I do agree my work to be entered into a database for it to be compared with existing sources, where it will remain in order to enable further comparisons with future theses. This does not grant any rights of reproduction and usage, however.

This document was neither presented to any other examination board nor has it been published.

German: Erklärung der Autorenschaft (Urheberschaft)

Ich erkläre hiermit, dass die vorliegende Arbeit ohne fremde Hilfe ausschließlich von mir erstellt und geschrieben worden ist. Jedwede verwendeten Quellen, direkter oder indirekter Art, sind als solche kenntlich gemacht worden. Mir ist die Tatsache bewusst, dass der Inhalt der Thesis in digitaler Form geprüft werden kann im Hinblick darauf, ob es sich ganz oder in Teilen um ein Plagiat handelt. Ich bin damit einverstanden, dass meine Arbeit in einer Datenbank eingegeben werden kann, um mit bereits bestehenden Quellen verglichen zu werden und dort auch verbleibt, um mit zukünftigen Arbeiten verglichen werden zu können. Dies berechtigt jedoch nicht zur Verwendung oder Vervielfältigung.

Diese Arbeit wurde noch keiner anderen Prüfungsbehörde vorgelegt noch wurde sie bisher veröffentlicht.

19.05.2026 .....
Date, Signature

Abstract

The goal of this thesis is exploring different methods to estimate distance to cars in the image, comparing them in terms of speed, accuracy and consistency. The reason for pursuing the topic is autonomous driving systems becoming more wide spread and being deployed across a range of different hardware. A single camera approach was discussed in depth as a cheaper and faster alternative to using LiDAR or stereo vision.

A range of models was tested on the KITTI dataset in order to determine how well they perform on the real world data. DINOv3 based approach was delved into deeper than others due to its better performance and versatility of the base model. YOLO26 was also tested for the same goal along with other options.

Contents

1 Introduction and state of the art **1**
- 1.1 Object recognition . . . 1
- 1.2 Stereo vision . . . 2
- 1.3 Monocular depth estimation . . . 2

2 Literature overview **3**
- 2.1 Field overview . . . 3
- 2.2 Using real object size . . . 4
- 2.3 Using embedding . . . 4
- 2.4 Other approaches . . . 5
- 2.5 Overview . . . 5

3 Statement and Motivation of Research **5**

4 Description of the Investigation **6**
- 4.1 KITTI dataset and source data used . . . 6
- 4.2 System and hardware used . . . 7
- 4.3 VGG16 feature extractor . . . 8
- 4.4 YOLO26, DINOv3 embeddings based approach . . . 9
- 4.5 DINO patch embeddings based model . . . 10
- 4.6 Estimation with Depth Anything v3 . . . 11

5 Evaluation of the Investigation **12**
- 5.1 General trends . . . 12
- 5.2 VGG16 based approach evaluation . . . 12
- 5.3 YOLO26 and DINOv3 embeddings evaluation . . . 14
- 5.4 DINO patch embeddings based model . . . 15
- 5.5 Depth Anything v3 evaluation . . . 16
- 5.6 Results summary . . . 17

6 Conclusions **18**

7 Acknowledgments **18**

A Supplementary materials **21**
- A.1 Plots . . . 21

1 Introduction and state of the art

Depth estimation is not a trivial task, that has existed in computer vision for a while. This thesis explores using several strategies to estimate distance in autonomous cars with 1 camera. Its relevance is high for autonomous driving, that is seeing more and more adoption and tests in recent years.

Major challenges in this sphere is being able to balance accuracy, cost of physical devices and speed. Very precise methods like LiDAR, stereo vision can be expensive and struggle on long distances, while monocular vision has higher error rate as there are simply less cues that can be used. Therefore, recent papers have explored using monocular techniques like integrating depth estimation into object recognition, using real dimensions of the object or estimations based on feature extractor embeddings.

The following subsections dive into key subtopics and explain key terms that are important for understanding the topic and the problem at hand. This introduction is partially based on the book **Computer vision: algorithms and applications**[14] as it thoroughly covers described topics and defines a lot of key terms necessary for this thesis.

1.1 Object recognition

Object recognition as a field aims to find bounding boxes of a given set of classes from the training data and to correctly classify the objects within those boxes. It has seen a lot of development in the past 20-30 years. Early methods were largely based on feature extraction and prediction based on such features. However, in recent years it is the deep learning approach that is gaining the most traction.

The main 2 models that are discussed in this thesis are YOLO[11] and RF-DETR[12] based on DINOv3[13]. YOLO(You only look once) is a single stage model that has traditionally been based on convolutional neural networks, but some later version integrate attention into the architecture. It can be used for real time object detection and has several sizes to match different complexities of devices it can be hosted on. Currently the latest available version is YOLO26[8], which is the one being used.

The YOLO family of models has been extensively explored due to being accessible for researchers and developers. It is deployed widely across different domains from autonomous driving to use in factories with many adjustments made to make it fit specific goals. Ultralytics documentation and website, while not perfect provide thorough documentation of the models and their use cases.

The same cannot be said for RF-DETR, whose docs don't cover nearly as many use cases and don't offer nearly the same number of examples of how to deploy and run the model. But this can be attributed to the project being relatively new and research oriented.

The other model used in this project is DINOv3[13] which is the base of RF-DETR and is

classified as self-supervised learning model. It is developed by Meta and has many use cases that include depth estimation. It provides a way of getting embedding for whole images as well as specific patches withing an image, which can later be used for depth estimation, classification and etc.

Both DINOv3 and RF-DETR use transformer architecture which sets them apart from YOLO in that regard. As shown in paper **Dist-YOLO: Fast Object Detection with Distance Estimation**[15] it is possible to modify these models not only to tune them to a specific use case, but also to predict more data, than initially intended. Authors of this particular paper modified it to estimate distance, which also increased accuracy of the model.

1.2 Stereo vision

The goal of stereo vision is to obtain depth information from a set of 2 images of the same scene. When distance between cameras and their intrinsic properties are provided it is possible to quite accurately compute distance to each point of the scene.

The base principle for this method is far away objects displacing very little compared to those close to the camera. So, if one can match the same points in both images, compute the disparity, then using intrinsic camera properties and distance between cameras it is possible to estimate the depth with high precision.

As mentioned before this method comes at a cost. 2 cameras instead of one means material costs. The processes of rectification, calibration and matching similar points can be computationally complex and requires a lot of knowledge about the setup. Moreover, it depends on the stereo base and distance up to which it can reasonably calculate how far away the objects are is limited.

1.3 Monocular depth estimation

Being able to estimate distance to an object with 1 camera does mean higher uncertainty, while offering computational and material savings. So, variety of models and approaches has been proposed in order to solve this problem.

Traditional approaches can be quite diverse. For example, capturing the same location during a motion of the camera. For instance, car moving forward. Also, the focus level of different objects might be used to infer depth. This relied on properties of the camera and light. Other visual features and clues can be used for this purpose.

Modern research largely seems to focus on applying machine learning to the problem. This lead to large assortment of model architectures and training approaches. This thesis focuses on a few of them. Depth estimation models range in architecture and type of data they predict. The few main types are listed below.

Metric distance in the classification of the model usually means that distance predicted is in meters and the goal when training was to estimate it exactly. This is the type of model this thesis focuses on.

Relative distance models focus on estimating how far away objects are in relation to each other. For instance, one car might be closer or further than the other and the model's goal is to decide which case is it.

Object specific vs non specific distance. Another question when analyzing a given model and its use cases is whether it estimates distance to every pixel in the frame or to exact objects. The models, papers and questions raised in this thesis are mainly concerned with object specific distances.

However, both approaches can coexist. For instance, in **Vehicle Distance Estimation from a Monocular Camera for Advanced Driver Assistance Systems**[9] the authors used a depth estimator to predict distance to each point. But then its outputs are used as inputs to the final neural network that estimates exact distance to an object within a box.

For the purpose of this thesis state of the art model Depth Anything v3[10] was used in order to predict metric distance in the image. It is based on DINO and has other use cases, but for the purpose of this project its depth estimation from a single image was explored the most.

2 Literature overview

This sections covers the most significant and influential papers that I read during my thesis work. These sections also allow for clearer understanding of differences between papers as names alone can be found confusing and quite similar across papers.

2.1 Field overview

Computer vision and object recognition has found a lot of applications in automotive industry. As paper **Analyzing Real-Time Object Detection with YOLO Algorithm in Automotive Applications: A Review**[4] explores in detail.

My thesis is mostly focused on distance estimation as one of the major problems in the realm of autonomous driving. Thus, sources that are presented next are generally trying to solve this problem in different ways. Not all of them however set that exact goal. Some focus more on understanding object recognition and distance estimation with deep learning and its specifics. For example, the paper **Visualization of Convolutional Neural Networks for Monocular Depth Estimation**[7] focuses on understanding how CNNs predict depth and what features allow them to do so. The authors constructed a system of several networks with the goal of obscuring as much of the image as possible, while

retaining accuracy or not seeing major drawback.

Other researchers focus on exploring unsupervised learning when applied to depth estimation. In paper **Unsupervised Depth Estimation From Monocular Images For Autonomous Vehicles**[5] the authors set out to do exactly that on the KITTI dataset as the training data.

2.2 Using real object size

Some papers use real object size or its approximation in order to estimate distance to a given object. Often using intrinsic properties of the camera is required in such approaches as well as knowing object average or exact size.

While I see this is a viable approach, the main question and hypothesis I have here is how well does the model work on different hardware and with different vehicles. Cars can come in all shapes and sizes, so using something like the average obscures a lot fo intra class variation. Moreover, when using height and width in pixels it can fall short in cameras with significantly higher or lower resolution.

For instance **DisNet: A novel method for distance estimation from monocular camera**[6]. This papers focuses on obstacle detection for rail transport. Authors cite that one of the main reasons automating rail transport is lagging behind cars is being able to detect objects, that can obstruct the way, at reasonable distances. Authors integrated state of the art YOLO model as the first step. It is used to find bounding boxes and to classify the object. The second part of model uses inverted height, width and diagonal along with average height, width and diagonal for selected class as inputs to disnet.

Another example of this approach is **Enhancing Road Safety: Inter-Vehicle Distance Estimation Using Fine-Tuned YOLOv9**[1], where authors use object's real life size and intrinsic properties of the camera to compute the real life distance to the object.

2.3 Using embedding

Another notable direction explored in research is using features or embedding extracted from the image in order to estimate the distance. A great example of this approach is **Learning Object-Specific Distance From a Monocular Image**[16]. In this paper authors propose 2 novel architectures that they found to work well for distance estimation in the realm of autonomous driving. This paper is one of the most important for this thesis.

The first version proposed works as follows. There first features are extracted using a CNN feature extractor. Results are then put through ROI pooling layer in order to get fixes size feature vectors. Finally, the vectors are used as inputs to a distance regressor that contains 3 fully connected layers(2048, 512,1) and soft plus function as the final layer. A more sophisticated version of this architecture is also proposed.

An important note here is that the authors trained all parts of the model together without separate training for feature extractor and distance regressor. I think that this means similar approaches that use a pre-trained feature extractor might require more complex architecture for the distance regressor. This is because not all features are equally important for distance estimation and object detection. So, one might need to make more effort to predict distance if embedding from an object recognition model is known.

2.4 Other approaches

One interesting way of finding distance is modifying YOLO or other object recognition models. For example, in **Dist-YOLO: Fast Object Detection with Distance Estimation**[15] There authors modify the architecture of the YOLO model by adding distance as one of the target values the model has to predict. Authors explore class aware and class agnostic distance prediction, with class agnostic predictions being superior. The exact error rates achieved in this research is **2.5m and 11%** when tested on KITTI dataset.

Also, using a separate depth estimation model is possible as explored in **Vehicle Distance Estimation from a Monocular Camera for Advanced Driver Assistance Systems**[9]. Their project consists of 3 parts: transformer-based object recognition model, transformer-based depth estimator and a distance predictor. The first 2 are run in parallel with the resulting information being fed to the distance regressor, that actually predicts the final distance to an object.

2.5 Overview

There are a lot of approaches in the field of computer vision to estimate depth. Among those researched recently I am mostly fascinated by the one proposed by the paper **Learning Object-Specific Distance From a Monocular Image**[16] and based my work largely on it as well as papers: **Vehicle Distance Estimation from a Monocular Camera for Advanced Driver Assistance Systems**[9] and **Visualization of Convolutional Neural Networks for Monocular Depth Estimation**[7].

3 Statement and Motivation of Research

The main question to be asked here is how fast and accurately can one predict distance to cars with modern tools and models. The main challenge here for me was understanding a variety of already existing approaches and finding one that I can see how to extend or improve in some way.

My goal in this project is to find how viable is it to use existing techniques and approaches to find object specific distances and how do they compare across a set of parameters. For instance, the use of embeddings from models like DINOv3 and YOLO26n in order to estimate depth to specific objects in the frame. I wanted to find out how it compares to other approaches and how fast it would be on mid-range modern hardware when used for inference.

There are reasons I wanted to try using existing models such as YOLO. Firstly, it is because I don't have the hardware or knowledge to build a complex solution for the problem from the ground up. Secondly, the fact that if it works it can provide a solid foundation for the system. After all these models are easy to deploy, well tested and provide way more info, that can be useful for different goals such as further analysis of what is in the frame or other machine learning models. Moreover, this allows for an easier upgrade of the system when a new version comes out so as to allow for the accuracy to further increase.

Another major goal for this project is to compare the approaches and to see how do they fall in comparison with one another and what would be the optimal solution for a given situation.

The questions for each one of them that I intended to ask was how accurate are they when applied to KITTI dataset data? How consistent are the results across the depth ranges? For example, do far away objects maintain the high accuracy score or not? What resources hardware and time wise does it take to train and run these models?

To be exact, the research questions were what approach can achieve the lowest MAE and RMSE scores on the KITTI dataset when detecting cars? What are the distributions of this accuracy when applied to objects from different parts of the depth range? In addition, how long does it take to infer the depth when presented with a single colored image from the dataset?

4 Description of the Investigation

4.1 KITTI dataset and source data used

Figure 1: Sample images from the KITTI dataset stacked together

Due to being a staple of computer vision research and being used in a lot of papers KITTI dataset was picked for testing the hypothesis and building the models on. This allows for easier comparison with other already published papers and reproducibility of this thesis should it be wanted.

We used KITTI[2] dataset in order to obtain ground truth distances to objects, their bounding boxes and the images. It was downloaded through the PyTorch library tools(Figure 1). To limit the scope of the project and reduce the variance of factors that can affect the

final depth predictions ground truth bounding boxes were used in order to obtain a set of cropped images each containing a car matched to a single depth value.

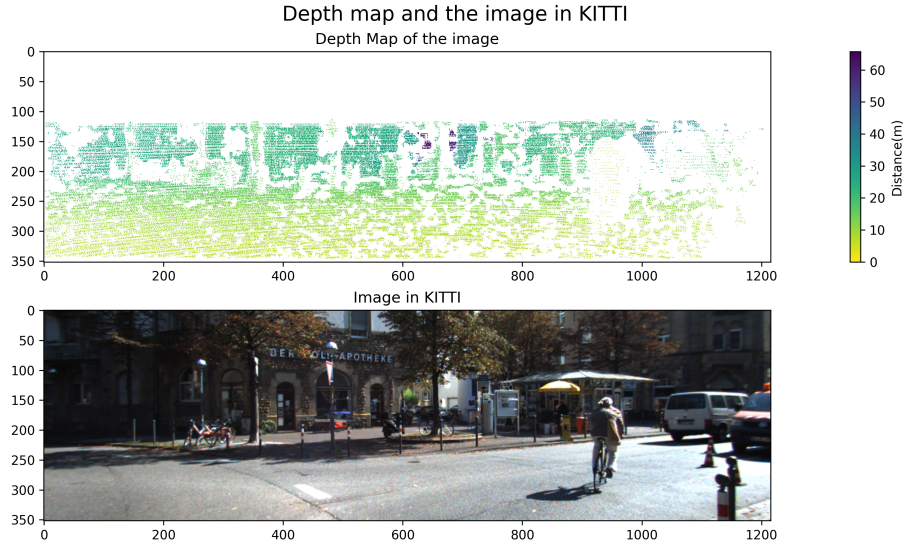

Figure 2: Sample image from the KITTI dataset

Another approach was also implemented and partially explored. A different version of KITTI[3] was downloaded from the official website of the project. It unlike the previous one doesn't contain labeled bounding boxes but provides the LiDAR image and the dev tools to turn it into depths array that can be indexed and processed(Figure 2).

The main advantage here is that this would allow with small adjustments to apply the code to a different dataset even if it is not fully labeled, but provides the depth and image data. The processing here required using YOLO26 to recognize the objects on the image from the desired subset of classes and then using the bounding box and depth info to find a singular depth value corresponding to the object by linearly fitting RANSAC within that box over the depth data. This way another smaller set of crops and depths was obtained that can be used in the latter stages of the project.

4.2 System and hardware used

This section is made to aid reproducability of the thesis work. The code was run on a local machine. It was a Lenovo laptop with an integrated intel GPU. Even though most libraries are most optimized for CUDA, intel XPU APIs were used due to this being the most available hardware to me at the time. However, it did take a considerable amount of time to ensure the GPU acceleration worked and that devices were recognized.

Exact system specifications can be found below:

```
OS: Fedora Linux 43 (KDE Plasma Desktop Edition) x86_64
Host: 83JK (IdeaPad Pro 5 14IAH10)
```

```
Kernel: Linux 6.19.10-200.fc43.x86_64
Shell: bash 5.3.0
DE: KDE Plasma 6.6.3
WM: KWin (Wayland)
Terminal: konsole 25.12.3
CPU: Intel(R) Core(TM) Ultra 9 285H (16) @ 5.40 GHz
GPU: Intel Arc Pro 130T/140T @ 2.35 GHz [Integrated]
Memory: 30.83 GiB
```

The development environment of choice was VS code. Virtual Python3.12 environment was used in order to make managing of libraries easier and to make it possible to discard them if needed and perform a reinstall. It should also be noted that the power setting was on performance mode during all the tests.

4.3 VGG16 feature extractor

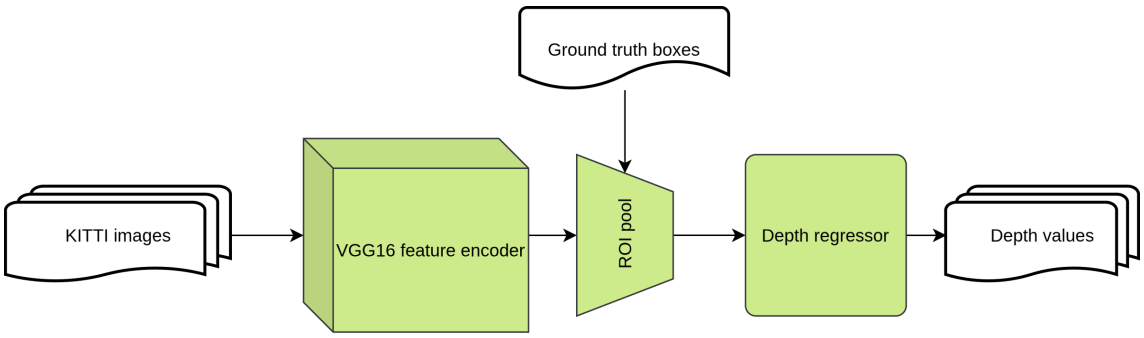

Figure 3: VGG16 based model structure

The first approach is based on the paper **Learning Object-Specific Distance From a Monocular Image**[16]. It employed training both a feature extractor and a depth regressor at the same time.

Even though the authors did not provide the code, I managed to recreate the model used for training as closely as possible using the PyTorch vgg16 feature extractor with default weights, that were not frozen, and a simple depth regressor(Figure 3). Authors said that they trained all parts of the model at the same time so I followed the same approach.

The reasoning I followed here was that not all features are equally important to understand depth. As the paper **Visualization of Convolutional Neural Networks for Monocular Depth Estimation**[7] pointed out certain parts of the image can be more crucial than the others. Namely borders, regions around the vanishing point and inner spaces of objects. Thus, performing back propagation through the feature extractor as well as the depth regressor can give higher weight to most relevant and crucial aspects of the image. So, even not the most recent vgg16 feature extractor can be quiet good provided the time and computational resource.

In order to be able to fully run and train it on my hardware I limited my reconstruction of the paper to the basic model excluding the classifier as it is outside the scope of this project. Ground truth bounding boxes were used as input to the ROI pooling layer.

The final model architecture consisted of a vgg16 feature extractor, ROI pooling layer that took its outputs along with the bounding boxes and used output shape (2,2) to match the first linear layer. In the original paper this feature was omitted from description. Finally each objects feature vector are fed into a depth regressor consisting of 3 Linear layers(2048,512,1) and 3 ReLU with softplus function as the last layer. ReLU was added in hopes of enhancing the performance. The model was trained for 10 epochs which took approximately 10 hours on my laptop. It managed to achieve results comparable to other methods.

4.4 YOLO26, DINOv3 embeddings based approach

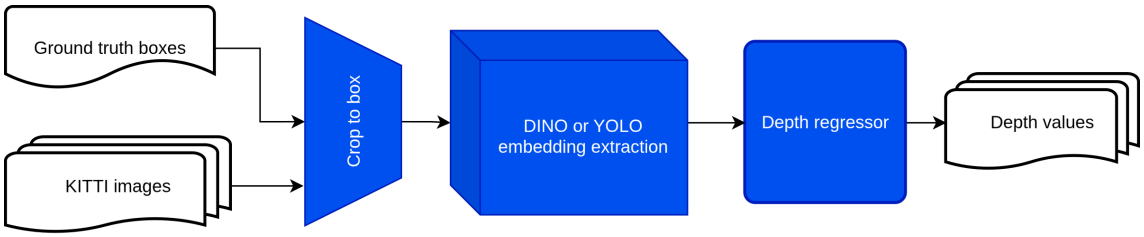

Figure 4: DINO and YOLO based model structure

This approach started from a question of is it possible to replace vgg16 with a more modern pre-trained model that could yield better results. The main models in question that were considered are YOLO26 and DINOv3.

YOLO is an object recognition model. However, its features might allow for depth estimation and, as one of the papers mentioned has shown, it is possible to adjust its architecture to make it predict depth among all other targets. Which in the study **Dist-YOLO: Fast Object Detection with Distance Estimation**[15] also increased the accuracy of the object recognition results.

In comparison, DINO family of models developed by Meta is not targeted towards object recognition and is in fact self-supervised learning model. Its official website advertises many use cases for its embeddings. It lists image segmentation, object recognition, depth estimation and etc. This made me pick the model for the experiments.

Here the ground truth boxes were used to crop the images to a specific object(Figure 4). Then each crop is passed through either YOLO26n or DINOv3 model to obtain an embedding vector that is stored for later processing. Unlike for VGG16 there is no pooling layer as embeddings are obtained directly for a specific object of the image. Final stage here was the depth regressor model.

To be specific, the model was comprised of 3 stages each being a Linear layer followed by batch norm of the same size and ReLU as well as 0.2 Dropout. The sizes are: 2048,1024,512 with of course the first layer input matching the type of vector used. The final stage is a single linear layer taking 512 input and producing a single value that is passed to softplus function.

The differences in YOLO and DINO are quite relevant at feature extraction stage for several reasons. Firstly, the length of YOLO vector is 256 as opposed to the 384 for DINO. Then in order to retrieve embeddings the *pooler_outout* is used for DINO on the output object after passing the inputs through the model. For the YOLO based model it is also relatively simple with *embed method* being applied to the crops of images directly. Both these approaches allow to retrieve the fixed size vectors of crops of the image but align themselves with the specifics of the model they use.

The key assumption here is that since the feature extractor is not narrowly targeting depth estimation unlike the one trained with depth estimator, it might require more complex depth estimator network in the end than the approach explored before. This hypothesis was further tested and the results are described in the next section of the thesis.

4.5 DINO patch embeddings based model

As some of the readers might know, it is also possible to extract embeddings of specific patches in the DINO model as part of its architecture. This allows for a different way to extract object specific embeddings using said feature.

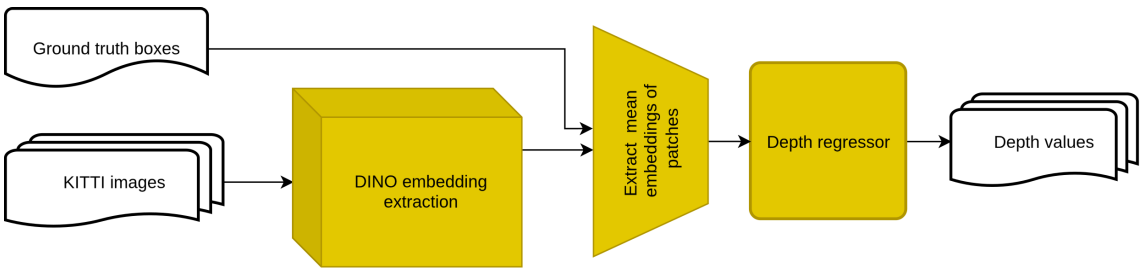

Figure 5: DINO model structure with patches

As can be seen on the picture(Figure 5) the main difference is the number of passes is less as we only perform a single forward pass on the image. This allows to extract patch specific embeddings as the very first step. They we later averaged over the patches with which the bounding box overlaps to extract object specific embeddings.

The way to extract the patch embeddings is using the *last_hidden_layer* property of the outputs object and indexing it. The key detail is that the patches that are produced have a certain size. Thus, in order to select the most relevant ones the coordinates of the bounding boxes are converted into the ranges for the patches to be selected and averaged. This means that it so no longer such a precise fit to the exact object dimensions that the coordinates in the full resolution image give.

Main advantage of this approach could be higher speed due to less passes over the image and possibility of more context and deeper understanding of the whole image. However, since patches are also way less precise than the exact bounding boxes in the approach discussed before there is a significant amount of noise being tracked. While this approach can improve performance, it might also worsen it. Further insights are described in the evaluation section.

4.6 Estimation with Depth Anything v3

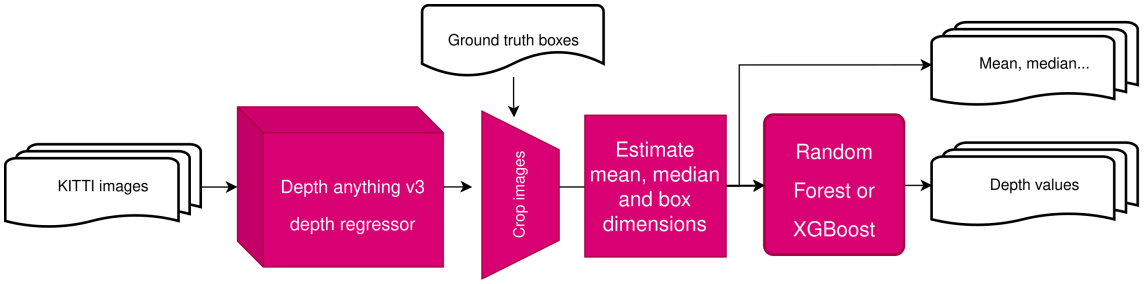

Figure 6: Depth anything v3 based model structure

This section aims to follow the strategy similar to **Vehicle Distance Estimation from a Monocular Camera for Advanced Driver Assistance Systems**[9].

Here the idea was to have a network focused on estimating distance to each part of the image(Figure 6). Following its inference the depth map can be cropped and its data can be passed to the final depth estimator, whose goal is to make the final value match closer to exact object distance. This is done to minimize effects of the background cropping into the bounding box and noise. The data that is gathered from the crop image is matched to the paper with the sole exclusion of the class. This is done due to the scope of the project being focused on cars.

To be exact the following inputs are gathered:

Name	Description
x1,y1	starting point of the bounding box
x2,y2	end of the bounding box
width, height	dimensions of the box obtained from coordinates
mean	mean depth value of the image within
median	median of the depth values
max	max depth withing the box
mean cut	mean with 10% cut on each side of the range

Table 1: Values passed to the final depth regressor obtained from depth image

The model used to estimate the depths for the entire image is Depth Anything 3[10]. It is a state of the art model based on DINO that has many applications such as video re-

construction, finding metric and relative distance to objects in the frame. It is used as the first stage of the process as it is more modern and readily available in place of the depth estimator from the paper.

It should however be noted that the depth map that it produces is lower resolution than the original image. As a result, when performing the crop to the object a conversion of bounding box coordinates in needed in order to ensure that it is indeed the object being selected and not something else. The scaling factor was computed as the ratio of the depth map dimension to the original image size along each axis.

When a vector with data specified in table 1 is retrieved it is passed to a final depth estimator. The paper compared several. So, just like the paper XGBoost Regressor and Random Forest Regressor were used. For each type of the model there were 2 versions: one with hyper parameters specified in the paper and the other with default ones and just a random seed to ensure consistency.

5 Evaluation of the Investigation

This sections covers the results achieved during this thesis. All models were tested on 8:2 training to testing split. More visualizations can also be found in the supplementary materials as some were excluded from the main text body.

5.1 General trends

One thing I noticed true for all models is that the error rate for close by objects is always lower than those further away. This does not directly correlate with the amount of data given for a range. For instance, the most common depths present were close to 20m away from the camera and not 0-10 as the error rate vs depth plots would suggest.

My hypothesis is that due to limited resolution it is simply easier to extract features from closer objects as they initially contain more data and detail. In that case better consistency could imply how well does a given model extrapolate when given less and less pixels for a given object.

Also, most models managed to achieve similar results, which could be attributed to a lot of them sharing similar architecture.

5.2 VGG16 based approach evaluation

The model managed to replicate the distribution of the training and testing data quite closely as can be seen in the picture(Figure 7). Moreover, the training loss of around 1.8m in MAE is comparable to other methods. Time to inference a single image is on average 0.1365s.

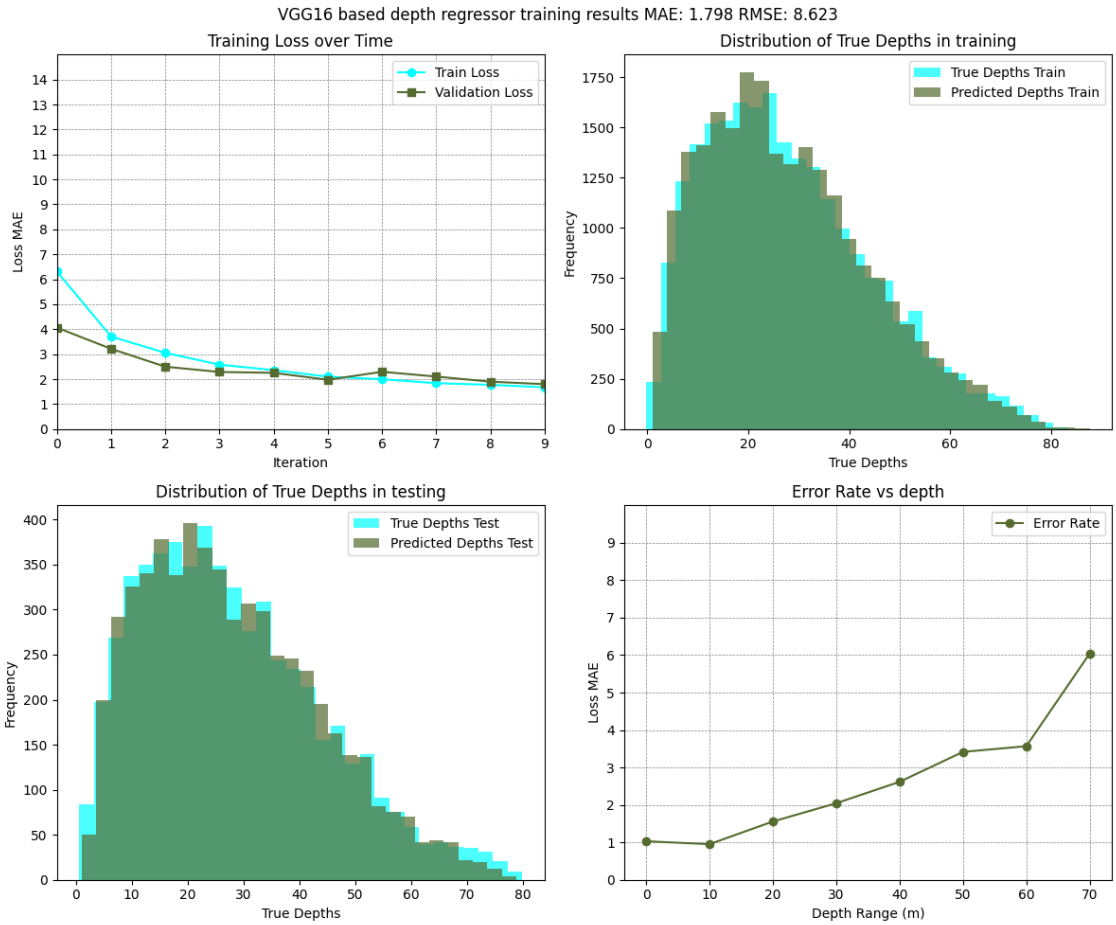

Figure 7: VGG16 based model performance, consistency and prediction distributions

Training duration

It is the number of epochs that draws my attention here. The model required only 10 epochs to achieve these results unlike DINO and YOLO embeddings based approaches that needed over 100 to get close to that range.

However, even though the epoch count is relatively small, the actual time needed to train this model is significantly larger. Compared to needing less than an hour to both extract the embeddings and train based on them, requiring 10 hours is a big gap. This is mainly due to feature extractor weights not being frozen and the back propagation being performed over the entire model and not just the depth regressor.

Consistency

Consistency was calculated by taking true depth values at range [10x:10x+10] for x in range (0,7) and computing the error rate for them. Results for each range were later plotted together for easier comparison. The consistency is better than YOLO, but worse than DINO based approach. It peaked at around 6m and lowest at 1m across the depth ranges.

5.3 YOLO26 and DINOv3 embeddings evaluation

This approach used YOLO and DINO embeddings retrieved from crops to object bounding boxes in the dataset. The model was trained for 300 epochs and achieved error rate close to 2m for both of them. However, there are things that set these 2 models apart.

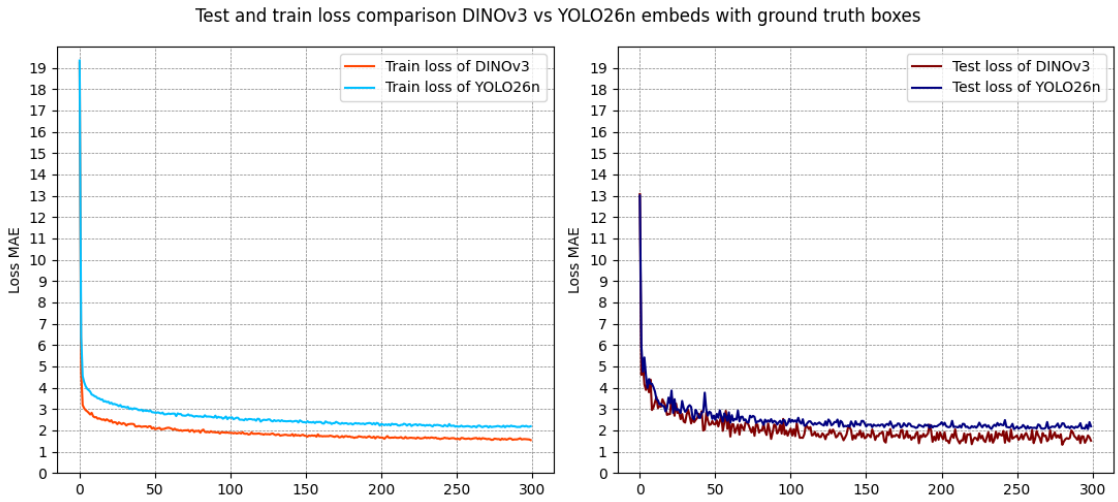

Figure 8: Comparison of YOLO and DINO based model's performance

While general, as can be seen in the image (Figure 8) both followed almost the same curve when it comes to decreasing the training and test loss over time, it should be noted that at each epoch DINO provided better results with close to 1m difference between them on testing dataset.

Training duration on both of them is relatively short as we only perform inference for each image once when retrieving embeddings and then the stored embeddings are used to train the model.

Consistency

By far the best consistency can be seen on the DINO based model (Figure 9). It manages to keep the MAE under 5m throughout the range. The same cannot be said for YOLO model, which reached almost 6m at highest.

Overall, DINO outperformed YOLO across the board. Thus, when possible DINO should be preferred for solving this problem of finding object specific depth, unless there is a clear benefit to extra data that YOLO provides with its results objects.

When it comes to speed both options are quite similar and exact time to extract embeddings and to predict based on them is almost the same as can be seem in (Table 3) and

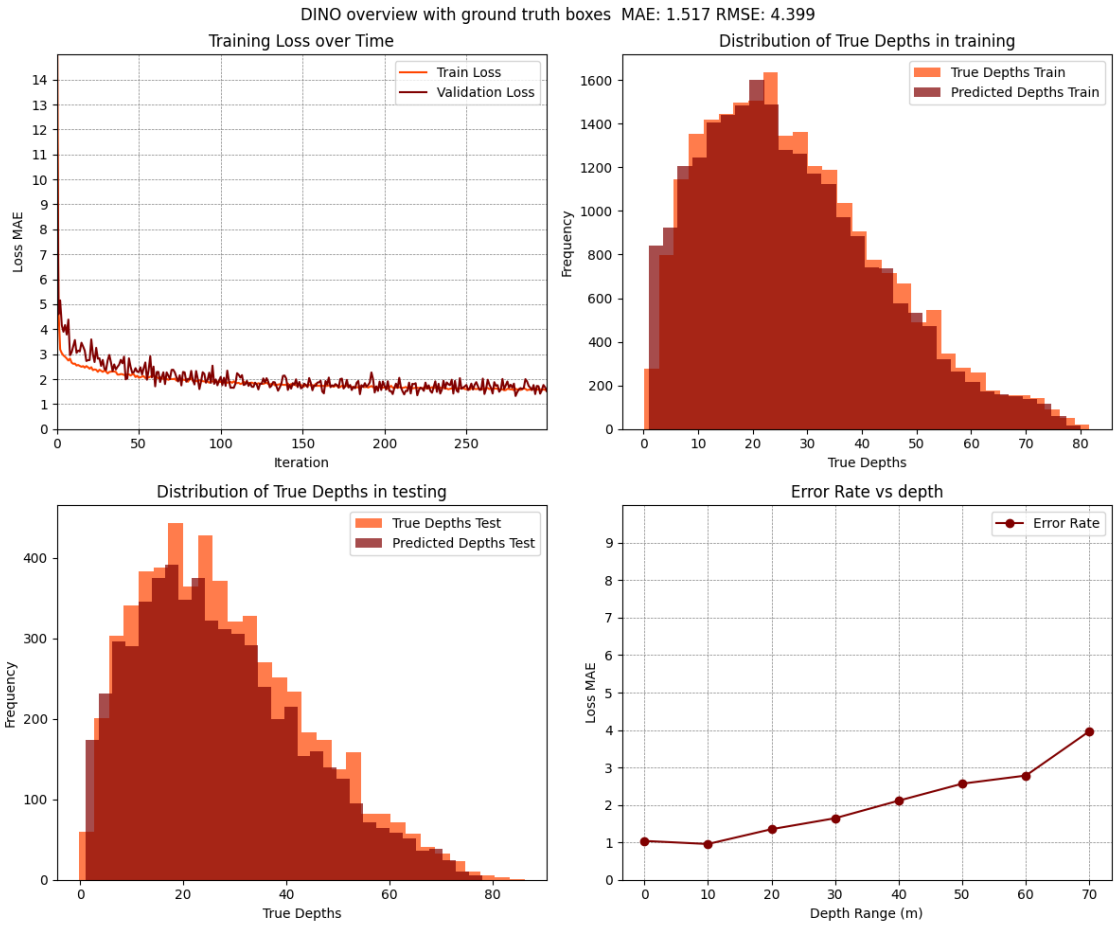

Figure 9: DINO based model performance, consistency and prediction distributions

on average it falls into under 0.1s per image range.

Depth regressor complexity here is greatly rewarded. When comparing the MAE scores of the more complex vs the depth regressor in the paper the YOLO model looses the most performance. This shows the power of DINO embeddings in terms of maintaining performance. Moreover, this confirms the hypothesis that since the original paper used a more integrated system it did not need a more complex depth regressor. This is compared to a less integrated solution with DINO that is not tailored to depth estimation that greatly benefits from the change.

5.4 DINO patch embeddings based model

Generally the results were subpar compared to all other methods as can be seen on the graph(Figure 12). Perhaps the boxes not being so precise introduce more noise than useful context into the numerical embedding of the object. However, the actual time to execute is significantly cut due to not needing to do several forward passes in DINO per image. Thus, the first approach of using DINO while computationally more intensive is

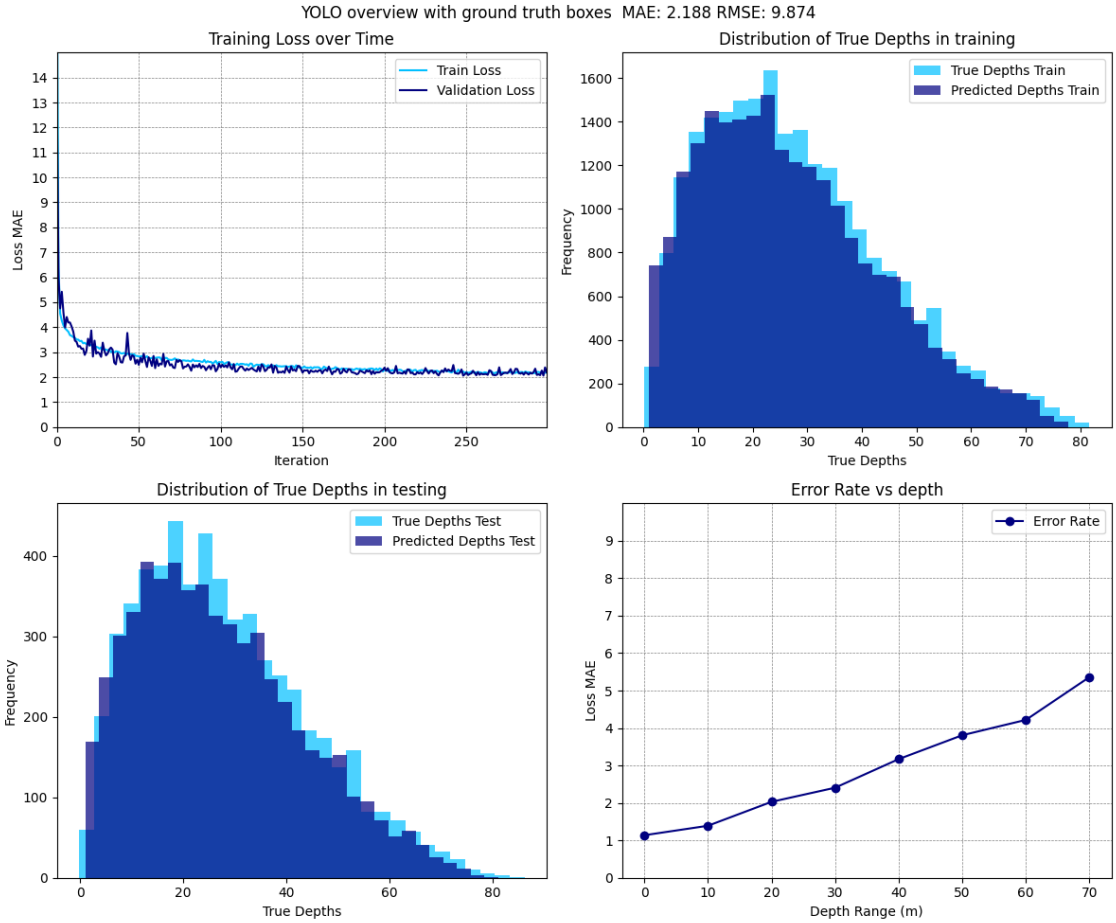

Figure 10: YOLO based model performance, consistency and prediction distributions

way more precise and fast enough that the cost is not noticeable.

5.5 Depth Anything v3 evaluation

Lastly, this model performed by far the worst in terms of error rates.

Firstly, I compared how close average, mean and mean cut values were to the true depth to the object. Generally it was around 4m which falls short of all models discussed so far (Table 2). Secondly, in order to follow the same strategy as the paper **Vehicle Distance Estimation from a Monocular Camera for Advanced Driver Assistance Systems**[9] the features gathered were passed to a random forest regressor and a XGBoost regressor models with exploring both using default and paper specific hyper parameters.

I expected the second model to decrease the shortcoming of the depth anything model and get closer to the true depths. Yet the disparity only increased with final MAE being over 18m, which is considerably worse then even just taking initial averages as can be seen in(Table 2).

Final model type	MAE(m)	Time(s)
XGBosst Regressor paper like	18.394	0.1031
XGBoost Regressor default	18.403	0.1031
Random forest regressor paper like	18.355	0.1033
Random forest regressor default	18.395	0.1032
Mean	4.134	
Median	4.448	
Mean cut	4.027	

Table 2: Depth anything based models accuracy and time to inference an image(avg). Also, includes MAE if raw mean and median are used.

To conclude, while I see the architecture proposed in[9] feasible and promising it would require more sophisticated or accurate depth regressor to outperform the DINO model.

I am not sure why the results got worse with the random forest and XGBoost instead of improving. Perhaps, the data extracted doesn't have clear patterns that can be learned from or the features chosen are not the most optimal solution. I find this question worth pursuing but outside the time and scope of this thesis.

5.6 Results summary

When put together the following becomes clear. Despite not being trained specifically for that purpose, DINOv3 feature vectors are flexible and contain enough information to predict depth consistently and accurately. Its exact comparison with other models can be seen in the table below (Table 3).

Model type	MAE(m)	RMSE(m)	Embedding(s)	Depth reg-r(s)	Time(s)
DINOv3s	1.517	4.399	0.09427	0.0001	0.09437
VGG16	1.798	8.623	-	-	0.1365
YOLO26n	2.188	9.874	0.07788	0.0001	0.07798
DINOv3s paper regressor	2.431	12.264	0.09427	0.0001	0.09437
YOLO26n paper regressor	3.905	28.106	0.07788	0.0001	0.07798
DINOv3s (patches)	4.188	36.544	0.02447	0.0001	0.02457
DINOv3s (p) paper regressor	6.321	82.417	0.02447	0.0001	0.02457

Table 3: Comparison of the models in embeddings based approach

Moreover, introducing a more complex depth regressor significantly improves performance for DINO and YOLO based systems since features extracted there are not nearly as focused on depth estimation as VGG16 model. In contrast, larger DINO models do not lead to much better performance, while requiring more time or better hardware to run properly. The hypothesis of YOLO features being applicable for depth estimation has proved to be true as even though it doesn't surpass DINO by any means it manages to

hold up well compared to other approaches.

The depth anything v3 based model exhibited sub par performance compared to other solutions. This is likely due to either the initial depth regressor not being nearly as tailored to road and car data or simply being less accurate. Another possible reason is perhaps that maybe extracted box dimensions and depths are simply not enough to make reasonable predictions if the depths aren't close to the ground truth to begin with.

Throughout the graphs another trend can be seen that was mentioned at the start of this section. The accuracy decreases with depth. The further away an object is the less accurate any prediction becomes, which is true for all models here. In addition, it should be noted that feature extraction is by far the most costly operation in the process. Most of the time of DINO and YOLO based models was spent on extracting features, rather than predicting depth. As a result, improvements to the depth regressor can yield better performance at almost no cost and speedup of the DINO model is the way to decrease execution time.

6 Conclusions

Embedding based approach has shown itself to be a performant and modular way to estimate distances. With modern tools like **DINOv3, YOLO26 and PyTorch** it is possible to accurately and consistently predict depth with a single camera at a reasonable pace. However, depending on the method and implementation used results can vary significantly. On one hand more general purpose feature extractors like DINOv3 or YOLO26 require more complex depth regressor for maximum accuracy and prefer using crops of images for embeddings. On the other hand fine tuning a less complex feature extractor like **VGG16** for depth estimation does not require nearly as much depth regressor complexity, but comes at way greater computational cost during training.

Overall all methods discussed manage to have sufficient speed to be used in real time applications. All models times were either under 0.1s or very close to that value allowing for **at least 8-10 frames per second for all methods discussed**. If further speed up is required parallelization of feature extraction between different crops should be ensured as it is the most costly operation of the process. Using larger models introduces slow down, but does not improve performance in a significant way that would make it a sensible strategy, but the key consideration here is the hardware available and where such systems are deployed.

7 Acknowledgments

I would like to thank professor Jakob Suchan for supervising this thesis throughout the semester and incredible feedback given that aided this project to such a high degree. Also, great amount of help was provided by Mr. Salim for code feedback and running the larger models as my own laptop was the limitation here.

References

[1] Muhammad Hamza Frooq et al. “Enhancing Road Safety: Inter-Vehicle Distance Estimation Using Fine-Tuned YOLOv9”. In: *2024 5th International Conference on Innovative Computing (ICIC)*. 2024, pp. 1–6. DOI: 10.1109/ICIC63915.2024.11116079.

[2] Andreas Geiger, Philip Lenz, and Raquel Urtasun. “Are we ready for Autonomous Driving? The KITTI Vision Benchmark Suite”. In: *Conference on Computer Vision and Pattern Recognition (CVPR)*. 2012.

[3] Andreas Geiger et al. “Vision meets Robotics: The KITTI Dataset”. In: *International Journal of Robotics Research (IJRR)* (2013).

[4] Carmen Gheorghe et al. “Analyzing Real-Time Object Detection with YOLO Algorithm in Automotive Applications: A Review”. In: *Computer Modeling in Engineering & Sciences* 141.3 (2024), pp. 1939–1981. ISSN: 1526-1506. DOI: 10.32604/cmes.2024.054735. URL: http://www.techscience.com/CMES/v141n3/58495.

[5] V Harisankar., Variyar V.V Sajith, and K P Soman. “Unsupervised Depth Estimation From Monocular Images For Autonomous Vehicles”. In: *2020 Fourth International Conference on Computing Methodologies and Communication (ICCMC)*. 2020, pp. 904–909. DOI: 10.1109/ICCMC48092.2020.ICCMC-000167.

[6] Muhammad Abdul Haseeb et al. “DisNet: A novel method for distance estimation from monocular camera”. In: *10th Planning, Perception and Navigation for Intelligent Vehicles (PPNIV18), IROS* (2018).

[7] Junjie Hu, Yan Zhang, and Takayuki Okatani. “Visualization of convolutional neural networks for monocular depth estimation”. In: *Proceedings of the IEEE/CVF international conference on computer vision*. 2019, pp. 3869–3878.

[8] Glenn Jocher and Jing Qiu. *Ultralytics YOLO26*. Version 26.0.0. 2026. URL: https://github.com/ultralytics/ultralytics.

[9] Seungyoo Lee et al. “Vehicle Distance Estimation from a Monocular Camera for Advanced Driver Assistance Systems”. In: *Symmetry* 14.12 (2022). ISSN: 2073-8994. DOI: 10.3390/sym14122657. URL: https://www.mdpi.com/2073-8994/14/12/2657.

[10] Haotong Lin et al. “Depth Anything 3: recovering the visual space from any views”. In: *arXiv preprint arXiv:2511.10647* (2025).

[11] Joseph Redmon et al. “You Only Look Once: Unified, Real-Time Object Detection”. In: *CoRR* abs/1506.02640 (2015). arXiv: 1506.02640. URL: http://arxiv.org/abs/1506.02640.

[12] Isaac Robinson et al. “RF-DETR: Neural Architecture Search for Real-Time Detection Transformers”. In: *The Fourteenth International Conference on Learning Representations*. 2026. URL: https://openreview.net/forum?id=qHm5GePxTh.

[13] Oriane Siméoni et al. “Dinov3”. In: *arXiv preprint arXiv:2508.10104* (2025).

[14] Richard Szeliski. *Computer vision: algorithms and applications*. Springer Nature, 2022.

[15] Marek Vajgl, Petr Hurtik, and Tomáš Nejezchleba. “Dist-YOLO: Fast Object Detection with Distance Estimation”. In: *Applied Sciences* 12.3 (2022). ISSN: 2076-3417. DOI: 10.3390/app12031354. URL: https://www.mdpi.com/2076-3417/12/3/1354.

[16] Jing Zhu and Yi Fang. “Learning Object-Specific Distance From a Monocular Image”. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. Oct. 2019.

A Supplementary materials

A.1 Plots

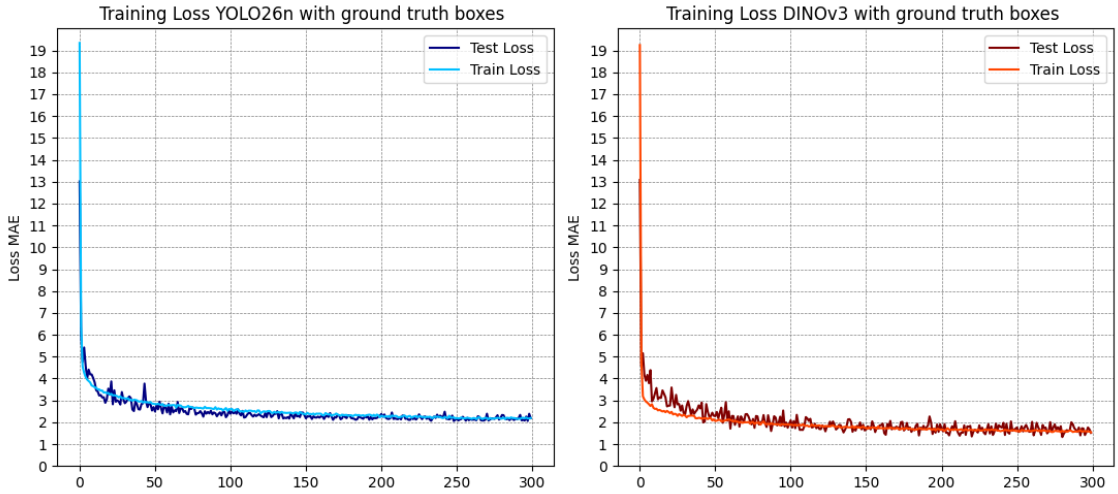

Figure 11: Comparison of YOLO and DINO based model's performance

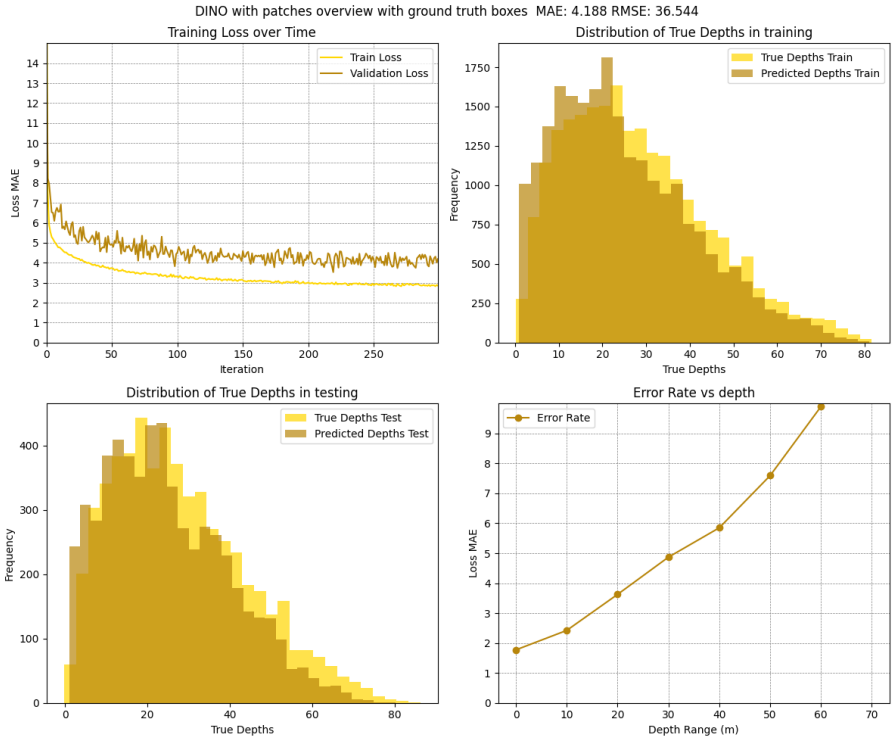

Figure 12: DINO patches based model performance, consistency and prediction distributions